Kristianstad University
SE-291 88 Kristianstad
+46 44-250 30 00
www.hkr.se

Assignment Lab 2

Remix Personal Portfolio
Show Your Work
(3 pointer)

Manuela Hackl

Manuela Hackl 0787 | manuela.hackl0787@stud.hkr.se

GitHub Repository: LINK

Coding Journey 3
HTML & CSS 3
Functions and Eventlisteners 4
The Email and Web validation 6
The topic selection and Data consent 7
The form submission 7
Challenges 8
One submit, multiple functions 8
Connecting the dots: DOM 9
Personal reflection 10
Which interaction are you most confident explaining, and why? 10
Visual documentation 11
Sources 15

Project Overview

In the course of Lab2, I extended my Lab1 portfolio website further into an advanced professional portfolio by introducing interactive Javascript functionalities from the course.

As a complete beginner in JavaScript, I aimed for 3 points and decided to focus exclusively on the newsletter/contact form.

This implementation uses Vanilla Javascript and further coding with HTML and CSS.

The improvements identified for Lab1 were not yet implemented. Therefore, the HTML and CSS used was based on the Lab1 Versions, submitted on 23rd September 2026. These improvements will be uploaded separately and integrated into The Lab 2 at a later stage as shown in the outlook.

Coding Journey

HTML & CSS

The contact form was implemented at the bottom of the sites index.html, portfolio.html and me.html in the corresponding section:

```
<section class="booking" id="section-meetme">
```

The About me page (me.html) was designed as the main touchpoint for website visitors to get in touch. Therefore, it shows the fully required feedback form as requested in Lab2.

The homepage (index.html) and Portfolio page (portfolio.html) contain a reduced contact form with the same functionalities, to focus on the informative content.

The visual design of the validation steps and error message is defined in the style.css file. Valid fields are indicated by a green border and green output messages. In contrast, invalid input triggers red text and red borders.

Example:

```
.errorborder, #message.errorborder {
   border: 2px solid red;
}
.errortxt {
   color: red;
}
.verifiedtxt {
   color: #1F3A52;
   text-decoration: bold;
}
.verifiedborder, #message.verifiedborder {
   border: 2px solid green;
}
```

Functions and Eventlisteners

The initial JavaScript code in “script.js” did not yet include functions and mainly concentrated on event listener elements and if-statements for validating form fields and updating visual design.

While developing script.js, the event listeners soon developed into functions to maintain and coordinate the repeating validation logic in an optimized manner. WHile the conditional statements (if/else/else if) remained essential for validation.

The main functions used in “script.js” are as follows:

```
validateFullName();

validateEmail();

validateWebsite();

validateTopic();

validateComments();

validateData();

resetForm()
```

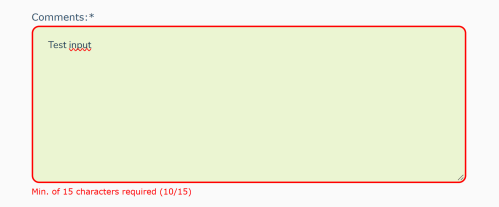
Each function is responsible for the validation of a specific input field in the feedback form, updating its status and showing visual feedback.

Event listeners were then used exclusively to respond to user inputs and trigger functions. For example, the comments counter is updated through the input event with immediate feedback while user enters or removes text:

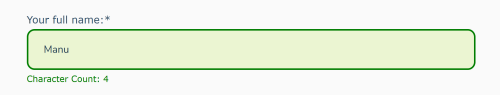
```
comments.addEventListener("input", validateComments)
```

The Name and Comment counters

Two character counters were implemented for the name and comments input fields. Both are updated dynamically with input event listeners with immediate feedback while the user types.



Picture: Comments counter showing incomplete input (0311_1.PNG)



Picture: Name counter showing valid input (0311_2.PNG)

The name field requires a minimum of 2 characters, while the comments input requires at least 15 characters.

In addition to validating the required amount of charters, a validation is also checking whether the name input contains permitted characters, such as international letters:

```
const validname = /^[\p{L}\p{M}\s'-]+$/u.test(textname)
```

Both fields change their status as defined in the css, depending on whether the minimum characters are met.

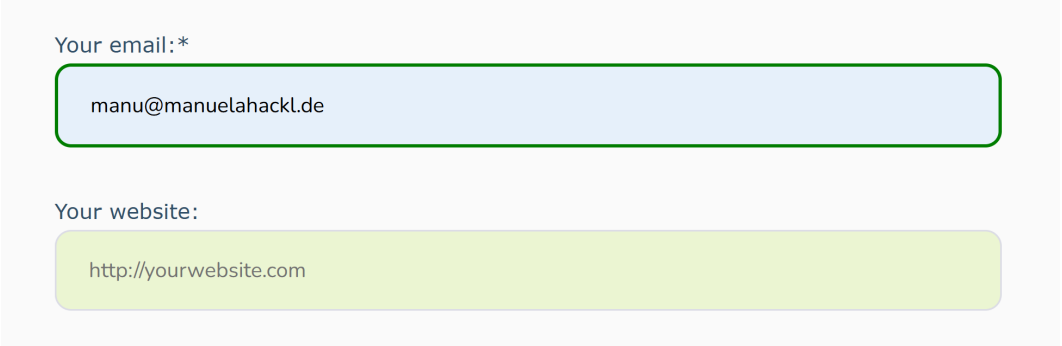
The functions validateFullName() and validateComments() are hereby managing the validation status and the visual feedback.

The Email and Web validation

The Email validation checks for plausible email structure, including an @ symbol and a domain with the browser's built in validation.

If the entered address is invalid, it again shows an error message with an error border around the field as defined in the css. In case of a valid input, it shows the corresponding success formatting.

The website field was defined as optional, so if it is left empty it will not output any error message or prevent the form from submission. Once a URL is inserted, the browser's URL validation is used to check the validity. To guide the user, a placeholdershows the correctway to inputthe URL. ValidateWebsite() hereby handles the status.



Picture: valid email output and website placeholder

The topic selection and Data consent

The topic selection, as well as the data consent are required fields.

The topic can be selected by a dropdown and the function validateTopic() checks if the option is valid. If the field remains empty, an error message is shown as feedback.

validateData() checks if the consent is given to process the data inserted in the required fields.

The form submission

When the form is submitted, the "event event.preventDefault()" is preventing the browser's default and prevents the page from reloading.

All validation functions are then executed and then the submission only proceeds if all required fields are valid, the website field is optional. However, if a URL is entered, it also must pass the browser builtin validation.

If all conditions are met, the page shows a personalized message the submitted user, using the output with “fullname.value”.

```
AllErrorMsg.textContent = "Thanks for your feedback, " +
fullname.value + "!";
```

In case of invalid or missing fields it outputs an error message, allowing the user to correct the input.

This provides immediate feedback and does not send an actual email.

Challenges

One submit, multiple functions

While implementing the first validation rules, I gradually became more confident writing the Javascript code and developed a routine setting up the remaining validation rules for the remaining fields.

However, a bigger challenge than expected was bringing them all together into one overall validation rule triggered by submission.

My initial approach was to use boolean states, such as nametrue = false, emailtrue = true. Those should be updated depending on the successful or failed validation of the corresponding field. I then used an if statement to check if all the boolean states were true.

Although this worked as a basic concept, I somehow struggled that all fields were checked properly and checked at the exact moment of submission.

After considering a loop, it took me several iterations of the submithandler, to arrive at a more structured solution and using functions to recall each dedicated validation function in a structured and clearer way when the form is submitted.

Each function returns its corresponding boolean state and the results are stored in constants before combining them in an overall if-statement.

This made the overall process clearer and easier for me to control, as I could see, whether all validation rules were fulfilled.

Another part of the learning process was understanding the difference between comparing a Boolean flag with true and calling a function that returns a Boolean value. I initially used explicit comparisons such as nametrue = = = true. As I moved towards a function-based structure, I learned that I could call the validation functions directly, store their returned values and use these results in the final condition. This made the overall process clearer and avoided relying solely on previously stored flag values.

Connecting the dots: DOM

One of the major challenges and biggest learning journey was understanding the relationship of HTML, CSS and javascript with each other through the DOM.

Initially, I found it difficult to connect the HTML elements with their corresponding JavaScript references and furthermore even more difficult to connect them with CSS classes for visual output.

For example, selecting the correct input field in JavaScript, referencing and importing the correct elementID and applying appropriate CSS to display the validation status required a clear understanding of how they work together. Thai was quite confusing at first and as a result, incorrect references and class assignments caused JavaScriptfeatures and visual feedback to work inconsistently or even not at all.

Personal reflection

Looking back at Lab2, I feel thatI have made a successful start with JavaScript. I was able to move from understanding basic concepts and writing simple code to building interactive form features using event listeners, functions and validation logic. Although I needed some guidance and several iterations to solve certain challenges, I gained a much better understanding of JavaScript and how it can be used.

Compared to HTML and CSS, I find it more complex. HTML and CSS seemed more intuitive for me , while JavaScript required a different way of thinking.

I rather see Lab 2 as a successful first step, rather than the point at which I have fully mastered it. Therefore, I still will need some lessons and practice to understand javascript further and recall or redo the implementations in the course of Lab2, but I am looking forward to building on this foundation and developing deeper understanding.

Which interaction are you most confident explaining, and why?

I probably feel most confident explaining the field validations using if statements and dedicated validation functions, as well as calling HTML and CSS properties.

I understand the basics of how the conditions check the users input against specific rules and how the result determines whether an error or success is displayed.

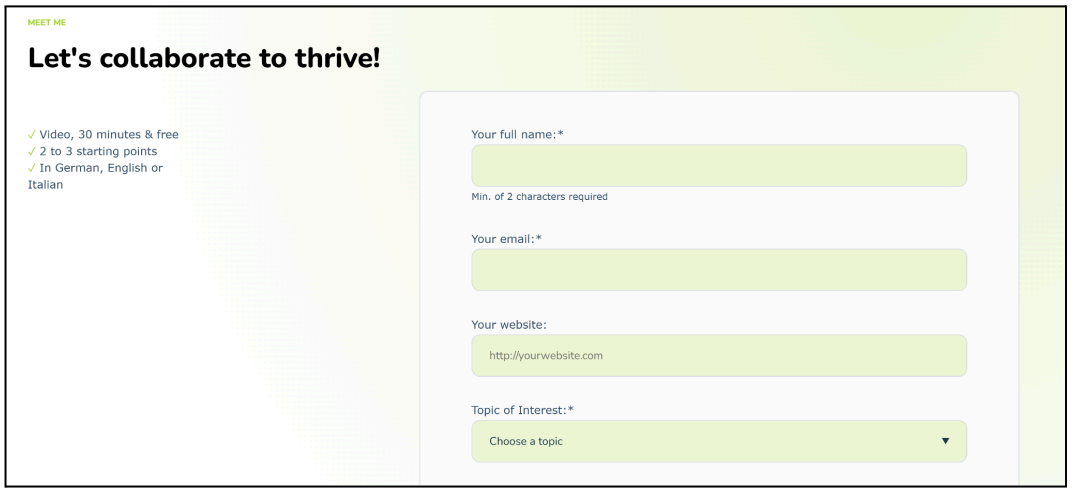
How does your Git history show the way your solution changed during development?

The development process involved several iterations, including the iteration from event listeners to functions, redefining the form feedback and submission validation. The GitHub commit history therefore not only reflects the technical development of the code but also my personal learning process and growing understanding of JavaScript.

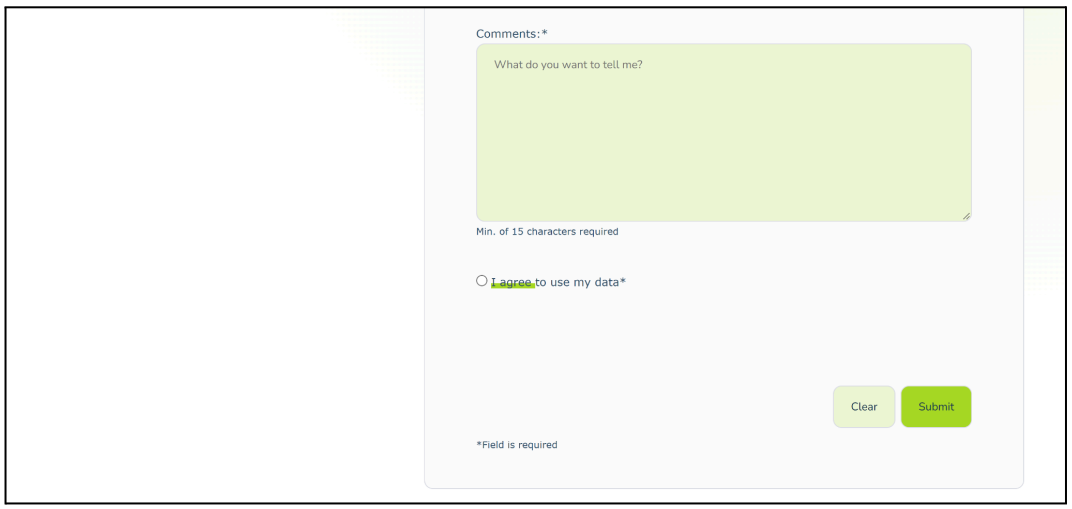
If you had one more development session, what would you improve or test further?

I would focus on further cross-device testing and aim for a possible 4 pointer rated website. Since developing the skills filter was a big goal of mine from the beginning, implementing it would be a big achievement for me.I would also use the possibility to improve my overall website experience with additional interactive features with javascript.

Visual documentation



Picture: Desktop mobile feedback form in normal state-part1: above the fold/upper part (0311_4_upper part.PNG)



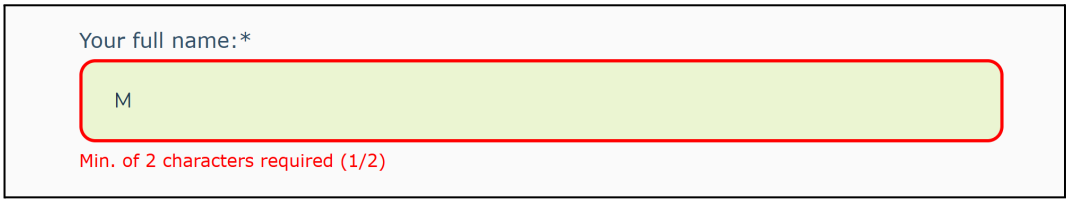
Picture: Desktop feedback form in normal state, lower below the fold/lower part (0311_4_btf-part.PNG)



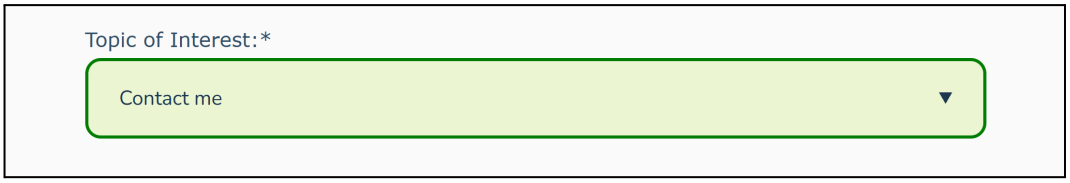
Pictures: mobile feedback form in normal state (0311_5_part1 until 0311_5_part4.PNG, from left to right)



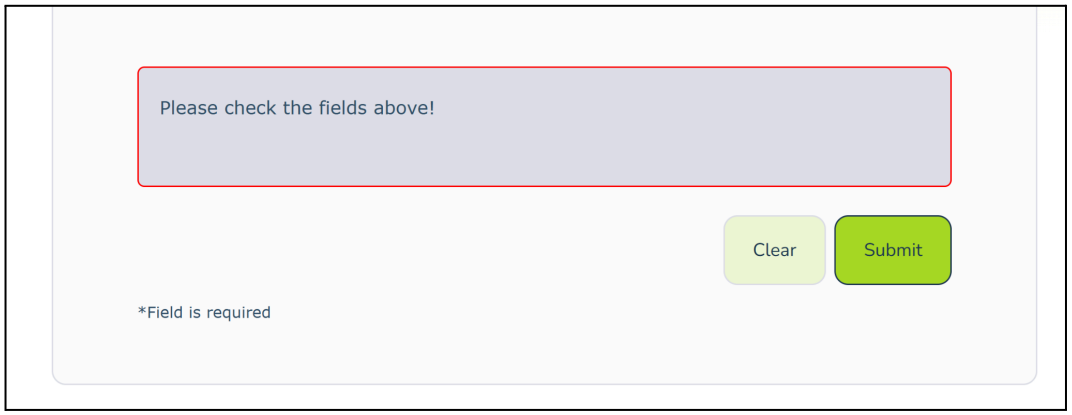
Picture: focus state (0311_6.PNG)



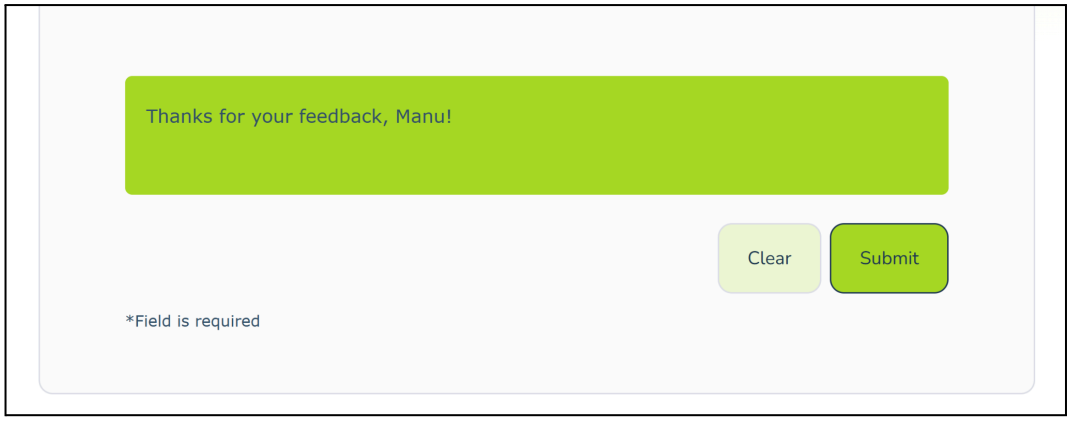
Picture: name field error with border and feedback message (0311_7.PNG)



Picture: exemplary output showing a valid input with border and feedback message (0311_8.PNG)



Picture: invalid field feedback upon submission (0311_9.PNG)



Picture: Successful submission feedback with username (0311_10.PNG)

Sources

- MSN, EventTarget: Methode addEventListener(), https://developer.mozilla.org/de/docs/Web/API/EventTarget/addEventListener

- MSN, <input type="reset"> HTML attribute value, https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/input/reset

- freeCodeCamp, How to Validate URLs in JavaScript, 22.11.2022: https://www.freecodecamp.org/news/how-to-validate-urls-in-javascript/